

TECHNICAL DOCUMENTATION

FinClaim

Confidential Claim Infrastructure for Receivable Finance — a permissioned, regulator-supervised registry that lets competing Bangladeshi banks and NBFIs check whether an invoice has already been pledged and how much financing room remains, without exposing client lists, pricing, or exact positions to one another.

Team DeluluChain — Meherun Mehnaj Miti, Md. Mahidul Islam Mahi, Khondaker Zarifa Haque, MD. Samiul Islam Tamim, Habibur Rahman, Faiyaz-Ur Rahman

Institution: United International University

Team ID: 6a8345f5ab2e0 · **Round:** BCOLBD 2026 Prototype Round

Repository: https://github.com/habib-153/BlockChain_FinClaim

1 Overview

FinClaim solves duplicate and over-collateralized receivable financing across institutions that do not trust each other with client-sensitive data. Bangladesh's Secured Transactions (Movable Property) Act, 2023 already mandates a national collateral registry for movable assets including receivables; FinClaim is a working implementation of that registry.

1.1 Core Capabilities

- **Legally-aware Receivable Claim Object** — distinguishes **pledge** (seller retains ownership, receivable used as collateral) from **assignment** (the right to collect is transferred), tracked as legally distinct on-chain events.
- **Confidential partial-financing check** — before approving a new claim, the ledger atomically checks `existing claims + new claim ≤ face value` and returns only the outcome and the new aggregate remaining capacity — never another lender's amount.
- **Linked-entity ("beneficial-ownership contagion") flag** — claims connected via shared directors, signatories, or addresses are flagged for **human review**, never auto-blocked; clearing a flag requires both a penalty fee (0.02% of the flagged invoice's value) and reviewer-verified documentation.
- **Supervisory consortium + public Merkle audit anchor** — Bangladesh Bank anchors the consortium without running it alone; a periodic Merkle root of ledger events is recorded so outside auditors can verify nothing was secretly altered, without seeing private transaction data.

1.2 Flagship Scenario

```
Receivable RCV-2026-04871 = BDT 1,000,000, buyer-attested, status ACTIVE.
```

```
> Padma Bank PLC requests 400,000 → approved. Capacity remaining: 600,000.
```

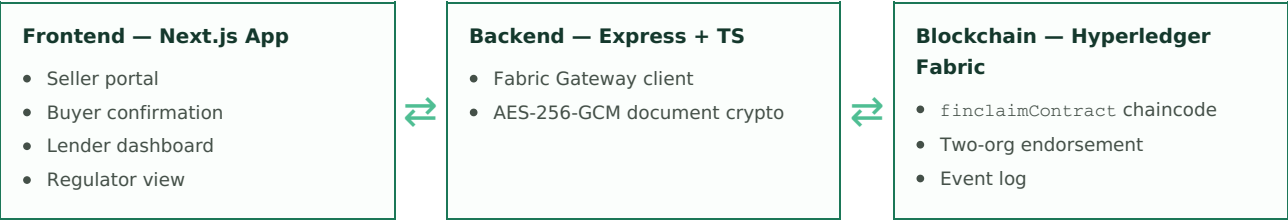
```
> Jamuna Capital & Finance Ltd. requests 300,000 → approved. Capacity remaining: 300,000.
```

```
> Meghna NBFI Ltd. requests 500,000 → rejected (exceeds remaining 300,000).
```

```
Meghna's own dashboard shows only "not eligible" — never Padma's or Jamuna's amounts.
```

Race-condition handling. Ledger transactions are ordered, not simultaneous. If, after Padma's 400,000 is approved (600,000 remaining), Jamuna's 300,000 and Meghna's 500,000 both individually fit within 600,000 but not together, whichever the network confirms first is evaluated against the live balance and approved; the second is then evaluated against the *already-updated* balance and rejected. The contract needs no special-case logic for this — it only ever sees one canonical ordered state.

2 System Architecture



2.1 Consensus Setup

Hyperledger Fabric on a two-organization channel topology:

Org	MSP	Role
Org1	Org1MSP	Lender Consortium (banks / NBFIs / fintechs)
Org2	Org2MSP	Regulator — Bangladesh Bank

The channel's endorsement policy requires **both organizations** to endorse state-changing transactions — the literal on-chain enforcement of the principle that no single institution, not even the regulator alone, can unilaterally write or alter claim state. Fabric's ordering service sequences transactions into blocks, which is what gives the "transactions are ordered, not simultaneous" guarantee the capacity-check logic depends on.

2.2 On-Chain vs. Off-Chain Split

Layer	Where	Contents
Actors & inputs	Off-chain	Seller portal, buyer confirmation, lender dashboard, regulator view (frontend/)
Processing & middleware	Off-chain	API/gateway routing, identity & oracle checks (KYC/BIN/tax)
Core protocol	On-chain (Fabric)	Legally-aware claim object, atomic capacity check, linked-entity flag records, freeze/unfreeze governance, event log — <code>chaincode/src/finclaimContract.ts</code>
External services	Off-chain	AES-256-GCM encrypted invoice store, hash-commitment confidentiality service, public Merkle anchor record

2.3 Data Model

Entity	Fields
Receivable	<code>id</code> , <code>sellerId</code> , <code>buyerId</code> , <code>faceValue</code> , <code>invoiceHash</code> , <code>status</code> (PENDING→ACTIVE→FLAGGED/DISPUTED→SETTLED), <code>totalClaimed</code> , <code>flagReason?</code> , <code>createdAt</code>
Claim	keyed by composite (<code>receivableId</code> , <code>lenderId</code>): <code>amount</code> , <code>claimType</code> (PLEDGE ASSIGNMENT), <code>status</code> (ACTIVE REJECTED SETTLED RELEASED), <code>commitment</code> , <code>createdAt</code>
Lender	onboarding/offboarding record, regulator-controlled; active flag gates whether a lender identity may submit claims at all
LinkedEntity	<code>businessIdA</code> , <code>businessIdB</code> , <code>reason</code> — the contagion-flag record, regulator-written
MerkleAnchor	<code>rootHash</code> , <code>eventCount</code> , <code>periodLabel</code> — a regulator-recorded checkpoint of ledger integrity

3 Privacy & Security

3.1 Data Minimization, Not Obfuscation

Raw invoices, prices, and commercial terms never touch the ledger. The `Receivable` object stores only an `invoiceHash` (a fingerprint of the encrypted off-chain document), identifiers, status, and aggregate `totalClaimed` — enough to prove state and authorization, nothing that reveals business terms.

3.2 Off-Chain Document Encryption

`backend/src/services/crypto.ts` implements AES-256-GCM encryption for invoice documents before they leave the submitting institution, with the decryption key held outside the ledger. The stated production path is per-institution keys in an HSM/KMS — a roadmap item, not what is running today.

3.3 Confidentiality Between Competing Lenders

Enforced at two layers:

- **On-chain:** `RequestFinancing` returns only the caller's own approval outcome and the new aggregate remaining capacity — another lender's claim amount is never serialized into a response a competing lender can read. Reading another lender's specific claim (`GetMyClaim`) is scoped to the calling lender's own identity; reading all claims on a receivable (`GetAllClaims`) is regulator-only.
- **In the frontend:** each lender's dashboard queries only its own claims plus the shared aggregate capacity meter — Meghna NBF's dashboard shows its own rejected claim and the capacity meter, never Padma's or Jamuna's amounts.

3.4 Claim-Amount Commitment

Each claim carries a `commitment` field — `SHA256(amount || nonce)`, computed in `commitAmount()` — binding the amount for dispute-time reveal without exposing it on the ledger in the meantime. This is a hash-commitment stand-in for the target of Pedersen commitments and Bulletproofs-style range proofs; a full zk-SNARK confidentiality circuit is a longer-term production goal.

3.5 Identity-Scoped Access Control

Access control is identity-scoped, not role-labeled. The chaincode checks the calling identity's MSP (`ctx.clientIdentity.getMSPID()`) against `Org1MSP` (Lender Consortium) or `Org2MSP` (Regulator) before allowing state-changing lender or regulator actions — enforced by the network's cryptographic identity, not a checkbox in application code.

Residual risks, acknowledged rather than hidden. A forged or collusive buyer attestation can still create a bad receivable — no ledger can make false real-world data true; this is why buyer attestation is designed to eventually bind to Bangladesh Bank's e-KYC framework rather than a bare signature. Linked-entity flags can produce innocent false positives, which is exactly why they trigger human review instead of an automatic freeze.

4 Governance

4.1 Membership Governance

Lenders are explicitly onboarded/offboarded by the Regulator org (`OnboardLender` / `OffboardLender`) before they can transact; `RequestFinancing` checks the lender is both registered and `active` before evaluating a claim — the on-chain form of "only approved institutions can run participant nodes or submit financing claims."

4.2 Business-Network Governance — No Unilateral Action

The two-org endorsement policy means neither a lender nor the regulator alone can write state without the other organization's peer endorsing the transaction, on every state-changing call. Freezing or unfreezing a disputed receivable is restricted to the Regulator identity at the chaincode-permission layer, and still executes as a transaction subject to the channel's shared endorsement policy — matching the principle that Bangladesh Bank anchors the consortium but does not run it alone.

4.3 Fraud Containment Governance — Review, Not Auto-Block

`FlagLinkedEntity` records a flag; nothing auto-freezes a receivable when a flag is written — freezing is a separate, explicit `FreezeReceivable` action a regulator takes after review. Linked-entity signals can be innocent (shared registered agent, coincidental address), so an automatic block would risk unfairly freezing legitimate finance. The frontend's flag-clearing flow (`ClearFlagDialog`) keeps the "Clear flag" action disabled until **both** the penalty-fee acknowledgment (0.02% of the flagged invoice's value, pre-calculated) **and** a supporting document are provided — payment alone cannot clear a flag.

4.4 Technical Infrastructure Governance

No native coin, no public speculative token — the ledger entry is a registry record, not something an SME buys or trades.

4.5 Auditability

No institution can silently delete or rewrite a settled claim — corrections happen only through new signed events (`getHistoryForKey` in `GetReceivableHistory` exposes the full append-only history for a receivable to the regulator), and the Merkle anchor record exists specifically so a party outside the consortium can verify ledger integrity without needing access to private transaction contents.

5 Smart Contract Function Reference

Source: `chaincode/src/finclaimContract.ts`

5.1 Receivable Lifecycle

<code>SubmitReceivable(receivableId, sellerId, buyerId, faceValue, invoiceHash)</code>	Create a receivable in <code>PENDING</code> .
<code>AttestBuyer(receivableId, buyerId)</code>	Buyer confirms the obligation, moves it to <code>ACTIVE</code> .
<code>SettleReceivable(receivableId)</code>	Mark a receivable settled.

5.2 Confidential Partial-Financing Check

<code>RequestFinancing(receivableId, lenderId, amount, claimType, commitment)</code>	Atomically checks remaining capacity and approves or rejects a claim, returning only the caller's own outcome and the new aggregate capacity.
<code>GetReceivableCapacity(receivableId)</code>	Public aggregate capacity remaining.
<code>GetMyClaim(receivableId, lenderId)</code>	A lender's own claim only.
<code>GetAllClaims(receivableId)</code>	Full cross-lender breakdown, regulator-only.

5.3 Governance & Compliance

<code>OnboardLender / OffboardLender / GetLender</code>	Consortium membership control.
<code>FlagLinkedEntity(businessIdA, businessIdB, reason)</code>	Records a beneficial-ownership contagion flag for human review.
<code>FreezeReceivable / UnfreezeReceivable</code>	Regulator-only dispute handling.
<code>GetReceivableHistory(receivableId)</code>	Full append-only history, regulator-only.
<code>RecordMerkleAnchor(rootHash, eventCount, periodLabel)</code>	Regulator-recorded ledger-integrity checkpoint.

6 Implementation Status

Core Features

- ✓ Legally-aware receivable + claim data model (pledge vs. assignment)
- ✓ Atomic confidential partial-financing check
- ✓ Two-organization Fabric endorsement policy
- ✓ Merkle-anchored ledger integrity checkpoints

Security Features

- ✓ AES-256-GCM off-chain invoice encryption
- ✓ Hash-based claim-amount commitment scheme
- ✓ Linked-entity contagion flag, human-review-only clearing
- ✓ Regulator-only audit views

User Experience

- ✓ Seller portal for receivable submission
- ✓ Buyer confirmation flow
- ✓ Lender dashboard with live capacity/status
- ✓ Regulator view with full audit trail

On the Roadmap

- Live e-KYC / BIN / tax oracle integration
- Per-institution Fabric CA identities
- Pedersen commitments + Bulletproofs range proofs, then a full zk-SNARK circuit
- Broadcasting the Merkle anchor root to a public permissionless chain
- Validator Participation Bond for lender-run nodes
- Per-institution HSM/KMS key management

7 Project Structure

```
FinClaim/
├── frontend/           # Next.js + Tailwind + shadcn/ui demo application
│   └── src/
│       ├── app/        # App router pages
│       ├── components/ # UI + domain components
│       ├── hooks/      # React hooks
│       └── lib/         # Types, utils, fixture data
├── backend/            # Fabric Gateway client + invoice encryption service
│   └── src/
│       ├── fabric/     # Gateway connection and contract calls
│       └── services/    # AES-256-GCM crypto service
├── chaincode/          # Hyperledger Fabric smart contract (TypeScript)
│   └── src/
│       └── finclaimContract.ts
└── docs/               # Whitepaper summary, prototype plan, phase handoff specs
```

7.1 Repository

Full source, commit history, and setup instructions: github.com/habib-153/BlockChain_FinClaim